

Assignment 3 - Task Farming

Rasserismus (Rasmus), Bundamin (Benjamin), Asgerholiker (Asger)

January 2026

1 Write a functioning master-worker program

The code is listed at the bottom and is also appended to the assignment.

2 Master-worker program for HEP data processing

The code is listed at the bottom and is also appended to the assignment.

3 Strong scaling of HEP processing using SLURM

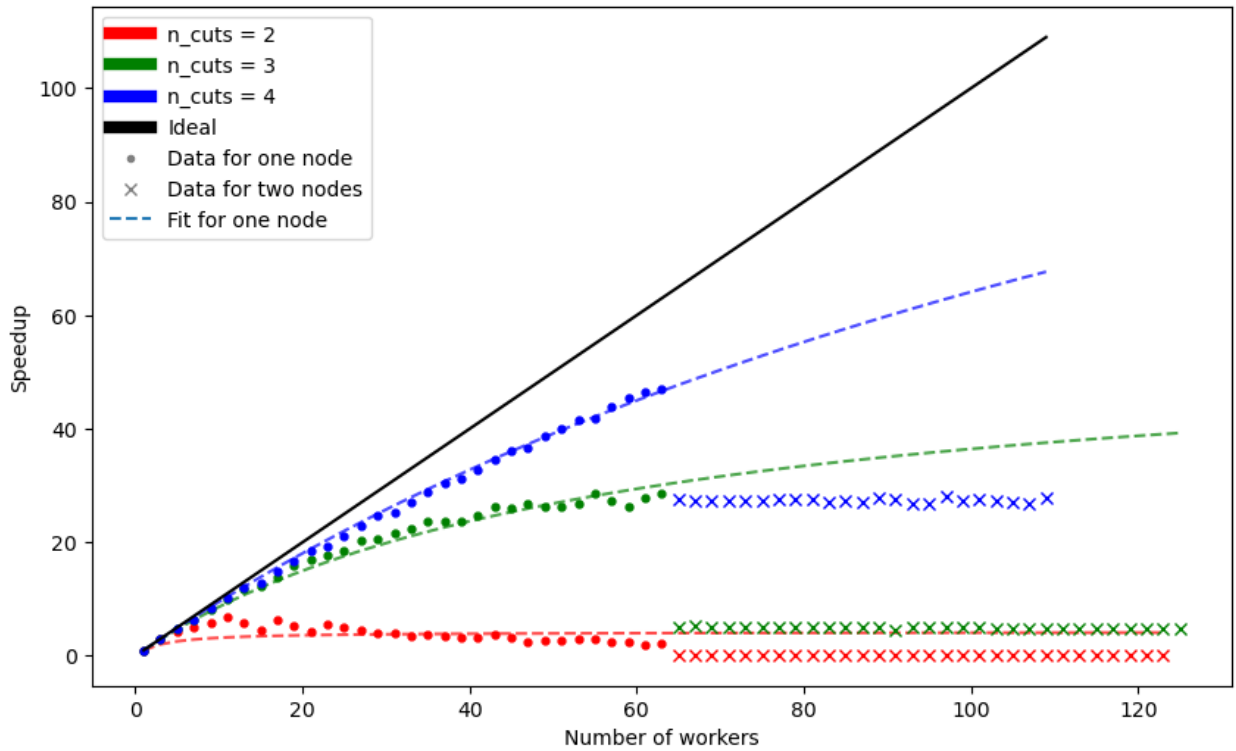


Figure 1: Plot shows the speedup for different n_{cuts} as a function of the numbers of workers. For every dataset we fit (dashed line) Amdahl's law to estimate the fraction of the code that can be parallelized. We also plot the "ideal" speedup for a code that is 0% serial.

3.1 Subtask a

In figure 1 we see the benchmarking of our code. We have run the code for $n_{cuts} = 2, 3, 4$. Increasing n_{cuts} increases the number of tasks to do, which increases the workload that can be done in parallel. We benchmarked on one node for 1-64 processors (the dots) and on two nodes (the x's) ¹. When using one node we see the speedup gained by adding more workers are higher for higher n_{cuts} . For $n_{cuts} = 3$ and $n_{cuts} = 4$ we see the speedup rise rapidly when adding more workers in the start, but going to larger number of workers we see we get diminishing returns. For $n_{cuts} = 2$ we see something weird, as we get an initial speedup when adding more workers until around 15 workers, where we actually get a decrease in performance, which is probably due to the low number of tasks and the increased time spend on communicating between worker and master. Looking at the data for two nodes we see that actually the performance is worse compared to the one node run. This is probably due to the communication between nodes being slower compared to the communication between processors in the same node. Important note: We used the $T_{wall}[1worker]$ from the 1 node run to calculate the speedup.

3.2 Subtask b

We fit Amdahl's law of strong scaling to our 1 node data in 1 to estimate the parallel fraction of the code. The fits can be seen in figure 1.

$$Speedup(\#workers) = \frac{1}{1 - P + P/\#workers} \quad (1)$$

For $n_{cuts} = 3$ and $n_{cuts} = 4$ the fit describes the data quite well, but not for $n_{cuts} = 2$, which seem reasonable as we get a decrease in speedup for high number of workers which adds additional overhead from the communication, which can't be captured by Amdahl's law. The fits gives the following parallel fractions:

n_{cuts}	2	3	4
Parallel fraction	0.76	0.97	0.99

Table 1: Estimated parallel fraction when fitting to the one node runs.

3.3 Subtask c

As mentioned before, our code is subject to strong scaling, as for a fixed n_{cuts} we get diminishing returns when adding more processors, and eventually we get no additional speedup. However we can increase the parallel fraction of our code by increasing the amount of work we should do. From table 1 the parallel fraction goes from 0.76 to 0.99 when increasing the number of cuts from 2 to 4. The bottlenecks in our code is most likely from the overhead that comes from communication, and the time the master uses to setup the cuts. The communication overhead is especially bad when using processors across different nodes, as we see clearly in 1. From the $n_{cuts} = 2$ case we saw that when the number of workers becomes comparable to the number of cuts, then the overhead from communication seem to cause a decrease in performance. If only doing n_{cuts} for the values we have tested 63 workers on one node is most optimal as the overhead from the communication between nodes kills the speedup gained by adding more workers. However if n_{cuts} is above 5, it could make sense to use multiple nodes, as the overhead is small compared to parallel part of the code.

4 Code

4.1 Simple Master-worker

¹We had some issues where the job couldn't run on all 128 processors, but doesn't matter as we don't expect anything different for the last few extra processors.

```

/*
Assignment: Make an MPI task farm. A "task" is a randomly generated integer.
To "execute" a task, the worker sleeps for the given number of milliseconds.
The result of a task should be send back from the worker to the master. It
contains the rank of the worker
*/

#include <iostream>
#include <random>
#include <chrono>
#include <thread>
#include <array>

// To run an MPI program we always need to include the MPI headers
#include <mpi.h>

const int NTASKS=5000; // number of tasks
const int RANDOMSEED=1234;

void master (int nworker) {
    std::array<int, NTASKS> task, result;

    // set up a random number generator
    std::random_device rd;
    //std::default_random_engine engine(rd());
    std::default_random_engine engine;
    engine.seed(RANDOMSEED);
    // make a distribution of random integers in the interval [0:30]
    std::uniform_int_distribution<int> distribution(0, 30);

    for (int& t : task) {
        t = distribution(engine); // set up some "tasks"
    }

    /*
IMPLEMENT HERE THE CODE FOR THE MASTER
ARRAY task contains tasks to be done. Send one element at a time to workers
ARRAY result should at completion contain the ranks of the workers that did
the corresponding tasks
*/
    int rank;
    int next_task = 0;
    int next_result = 0;

    //Send out initial tasks to workers
    for (int worker = 1; worker <= nworker; worker++){
        MPI_Send(&task[next_task], 1, MPI_INT, worker, 0, MPLCOMM_WORLD);
        next_task++;
    }

    //Keep sending out tasks
    while(next_task < NTASKS){

```

```

    //Receive result
    MPI_Recv(&rank, 1, MPI_INT, MPLANY_SOURCE, 0, MPLCOMM_WORLD, MPI_STATUS_IGNORE)

    //Determine worker
    result[next_result] = rank;
    next_result++;

    //Send out next task
    MPI_Send(&task[next_task], 1, MPI_INT, rank, 0, MPLCOMM_WORLD);

    //Move to new task
    next_task++;
}

//Receive final tasks and kill workers
int kill_val = -1;
for (int i = 0; i < nworker; i++) {
    MPI_Recv(&rank, 1, MPI_INT, MPLANY_SOURCE, 0, MPLCOMM_WORLD, MPI_STATUS_IGNORE)
    result[next_result] = rank;
    next_result++;
    MPI_Send(&kill_val, 1, MPI_INT, rank, 0, MPLCOMM_WORLD);
}

// Print out a status on how many tasks were completed by each worker
for (int worker=1; worker<=nworker; worker++) {
    int tasksdone = 0; int workdone = 0;
    for (int itask=0; itask<NTASKS; itask++)
        if (result[itask]==worker) {
            tasksdone++;
            workdone += task[itask];
        }
    std::cout << "Master:-Worker-" << worker << "-solved-" << tasksdone <<
        "-tasks\n";
}

}

// call this function to complete the task. It sleeps for task milliseconds
void task_function(int task) {
    std::this_thread::sleep_for(std::chrono::milliseconds(task));
}

void worker (int rank) {
    /*
    IMPLEMENT HERE THE CODE FOR THE WORKER
    Use a call to "task_function" to complete a task
    */
    while (true) {
        int task;

        MPI_Recv(&task, 1, MPI_INT, MPLANY_SOURCE, 0, MPLCOMM_WORLD, MPI_STATUS_IGNORE)
    }
}

```

```

        if (task == -1) {
            break;
        }

        task_function(task);

        //Send back to master
        MPI_Send(&rank, 1, MPI_INT, 0, 0, MPLCOMM_WORLD);
    }
}

int main(int argc, char *argv[]) {
    int nrank, rank;

    MPI_Init(&argc, &argv); // set up MPI
    MPI_Comm_size(MPLCOMM_WORLD, &nrank); // get the total number of ranks
    MPI_Comm_rank(MPLCOMM_WORLD, &rank); // get the rank of this process

    if (rank == 0) // rank 0 is the master
        master(nrank-1); // there is nrank-1 worker processes
    else // ranks in [1:nrank] are workers
        worker(rank);

    MPI_Finalize(); // shutdown MPI
}

```

4.2 High energy physics

```

/*
Assignment: Make an MPI task farm for analysing HEP data
To "execute" a task, the worker computes the accuracy of a specific set of cuts.
The resulting accuracy should be send back from the worker to the master.
*/

#include <iostream>
#include <sstream>
#include <fstream>
#include <iomanip>
#include <random>
#include <chrono>
#include <thread>
#include <array>
#include <vector>

// To run an MPI program we always need to include the MPI headers
#include <mpi.h>

// Number of cuts to try out for each event channel.
// BEWARE! Generates n_cuts^8 permutations to analyse.

```

```

// If you run many workers, you may want to increase from 3.
const int n_cuts = 4;
const long n_settings = (long) pow(n_cuts,8);
const long NO_MORE_TASKS = n_settings+1;

// Class to hold the main data set together with a bit of statistics
class Data {
public:
    long nevents=0;
    std::string name[8] = { "averageInteractionsPerCrossing", "p-Rhad", "p-Rhad1",
                           "p-TRTTrackOccupancy", "p_topoetcone40", "p_eTileGap3Cluster",
                           "p_phiModCalo", "p_etaModCalo" };
    std::vector<std::array<double, 8>> data; // event data
    std::vector<long> NvtxReco; // counters; don't use them
    std::vector<long> p_nTracks;
    std::vector<long> p_truthType; // authoritative truth about a signal

    std::vector<bool> signal; // True if p_truthType=2

    std::array<double,8> means_sig {0}, means_bckg {0}; // mean of signal and background
    std::array<double,8> flip; // flip sign if background larger than signal for type of
};

// Routine to read events data from csv file and calculate a bit of statistics
Data read_data() {
    // name of data file
    std::string filename="mc-ggH-16-13TeV-Zee-EGAM1-calocells-16249871.csv";
    std::ifstream csvfile(filename); // open file

    std::string line;
    std::getline(csvfile, line); // skip the first line

    Data ds; // variable to hold all data of the file

    while (std::getline(csvfile, line)) { // loop over lines until end of file
        if (line.empty()) continue; // skip empty lines
        std::istringstream iss(line);
        std::string element;
        std::array<double,8> data;

        // read in one line of data in to class
        std::getline(iss, element, ','); // line counter, skip it
        std::getline(iss, element, ','); // averageInteractionsPerCrossing
        data[0] = std::stod(element);
        std::getline(iss, element, ','); // NvtxReco
        ds.NvtxReco.push_back(std::stol(element));
        std::getline(iss, element, ','); // p_nTracks
        ds.p_nTracks.push_back(std::stol(element));
        // Load in a loop the 7 next data points:
        // p-Rhad, p-Rhad1, p-TRTTrackOccupancy, p_topoetcone40, p_eTileGap3Cluster, p-ph
        for(int i=1; i<8; i++) {
            std::getline(iss, element, ',');
            data[i] = std::stod(element);
        }
    }
}

```

```

        std::getline(iss , element , ','); // p_truthType
        ds.p_truthType.push_back(std::stol(element));
        ds.data.push_back(data);
        ds.nevents++;
    }

    // Calculate means. Signal has p_truthType = 2
    ds.signal.resize(ds.nevents);
    long nsig=0, nbckg=0;
    for (long ev=0; ev<ds.nevents; ev++) {
        ds.signal[ev] = ds.p_truthType[ev] == 2;
        if (ds.signal[ev]) {
            for (int i=0; i<8; i++) ds.means_sig[i] += ds.data[ev][i];
            nsig++;
        } else {
            for (int i=0; i<8; i++) ds.means_bckg[i] += ds.data[ev][i];
            nbckg++;
        }
    }
    for (int i=0; i<8; i++) {
        ds.means_sig[i] = ds.means_sig[i] / nsig;
        ds.means_bckg[i] = ds.means_bckg[i] / nbckg;
    }

    // check for flip and change sign of data and means if needed
    for (int i=0; i<8; i++) {
        ds.flip[i] = (ds.means_bckg[i] < ds.means_sig[i]) ? -1 : 1;
        for (long ev=0; ev<ds.nevents; ev++) ds.data[ev][i] *= ds.flip[i];
        ds.means_sig[i] = ds.means_sig[i] * ds.flip[i];
        ds.means_bckg[i] = ds.means_bckg[i] * ds.flip[i];
    }

    return ds;
}

// call this function to complete the task. It calculates the accuracy of a given set of
double task_function(std::array<double,8>& setting , Data& ds) {
    // pred evalautes to true if cuts for events are satisfied for all cuts
    std::vector<bool> pred(ds.nevents,true);
    for (long ev=0; ev<ds.nevents; ev++)
        for (int i=0; i<8; i++)
            pred[ev] = pred[ev] and (ds.data[ev][i] < setting[i]);

    // accuracy is percentage of events that are predicted as true signal if and only if
    double acc=0;
    for (long ev=0; ev<ds.nevents; ev++) acc += pred[ev] == ds.signal[ev];

    return acc / ds.nevents;
}

void master (int nworker , Data& ds) {
    std::array<std::array<double,8>,n_cuts> ranges; // ranges for cuts to explore

    // loop over different event channels and set up cuts

```

```

for (int i=0; i<8; i++) {
    for (int j=0; j<n_cuts; j++)
        ranges[j][i] = ds.means_sig[i] + j * (ds.means_bckg[i] - ds.means_sig[i]) / n
}

// generate list of all permutations of the cuts for each channel
std::vector<std::array<double,8>> settings(n_settings);
for (long k=0; k<n_settings; k++) {
    long div = 1;
    std::array<double,8> set;
    for (int i=0; i<8; i++) {
        long idx = (k / div) % n_cuts;
        set[i] = ranges[idx][i];
        div *= n_cuts;
    }
    settings[k] = set;
}

// results vector with the accuracy of each set of settings
std::vector<double> accuracy(n_settings);

auto tstart = std::chrono::high_resolution_clock::now(); // start time (nano-seconds)

size_t tasks_send = 0;

for (int i = 1; i < nworker + 1; i++){
    MPI_Send(&settings[tasks_send][0], 8, MPLDOUBLE, i, 0, MPLCOMM_WORLD);
    tasks_send += 1;

    //std::cout << tasks_send << " tasks_send\n";
}

size_t tasks_done = 0;
MPI_Status st;

while (tasks_done < settings.size()){
    MPI_Recv(&accuracy[tasks_done], 1, MPLDOUBLE, MPLANY_SOURCE, MPLANY_TAG, MPI_COMM_WORLD, &st);
    tasks_done += 1;

    if (tasks_done==settings.size()){
        break;
        std::cout << " All tasks done\n";
    }
    MPI_Send(&settings[tasks_send][0], 8, MPLDOUBLE, st.MPI_SOURCE, 0, MPLCOMM_WORLD);
    tasks_send += 1;
};

for (int i = 1; i < nworker + 1; i++){
    MPI_Send(&settings[tasks_send], 1, MPLDOUBLE, i, 1, MPLCOMM_WORLD);
}

auto tend = std::chrono::high_resolution_clock::now(); // end time (nano-seconds)

```

```

// diagnostics
// extract index and value for best accuracy
double best_accuracy_score=0;
long idx_best=0;
for (long k=0; k<n_settings; k++)
    if (accuracy[k] > best_accuracy_score) {
        best_accuracy_score = accuracy[k];
        idx_best = k;
    }

std::cout << nworker << ", " << (tend - tstart).count()*1e-9 << ", " << (tend - tstart)
}

void worker (int rank, Data& ds) {
    std::array<double,8> setting;
    int tag = 0;
    while(true){
        MPI_Status st;
        MPI_Recv(&setting[0], 8, MPLDOUBLE, 0, MPLANY_TAG, MPLCOMM_WORLD, &st );
        if (st.MPLTAG == 1){
            break;
        }

        double accuracy = task_function(setting, ds);
        MPI_Send(&accuracy, 1, MPLDOUBLE, 0, 0, MPLCOMM_WORLD);
    }
}

int main(int argc, char *argv[]) {
    int nrank, rank;

    MPI_Init(&argc, &argv); // set up MPI
    MPI_Comm_size(MPLCOMM_WORLD, &nrank); // get the total number of ranks
    MPI_Comm_rank(MPLCOMM_WORLD, &rank); // get the rank of this process

    // All ranks need to read the data
    Data ds = read_data();
    //std::cout << rank << std::endl;

    if (rank == 0) // rank 0 is the master
        master(nrank-1, ds); // there is nrank-1 worker processes
    else // ranks in [1:nrank] are workers
        worker(rank, ds);

    MPI_Finalize(); // shutdown MPI
}

```
